

Pandas Library

An Introduction to Data Analysis in
Python

What is Pandas?

- Open-source Python library for data analysis and manipulation
- Built on top of NumPy
- Provides data structures: Series (1D) and DataFrame (2D)
- Ideal for tabular, time-series and heterogeneous data

Core Data Structures

- Series: One-dimensional labeled array
- DataFrame: Two-dimensional labeled data structure
- Index objects: For labeling axes

```
import pandas as pd
```

```
# Create a Series
```

```
s = pd.Series([10, 20, 30], index=['a', 'b', 'c'])
```

```
print(s)
```

```
s.values
```

```
s.index
```

```
data = {  
    'Name': ['Alice', 'Bob', 'Charlie'],  
    'Age': [24, 27, 22],  
    'City': ['Delhi', 'Mumbai', 'Chennai']  
}
```

```
df = pd.DataFrame(data)  
print(df)
```

- **df.head()** → first 5 rows
- **df.tail()** → last 5 rows
- **df.shape** → (rows, columns)
- **df.columns** → column names
- **df.info()** → data types, null counts
- **df.describe()** → quick stats on numeric columns

Reading and Writing Data

- Read from CSV: `pd.read_csv('file.csv')`
- Read from Excel: `pd.read_excel('file.xlsx')`
- Save to CSV: `df.to_csv('file.csv', index=False)`

Selecting Data

- Select columns: `df['column']` or `df[['col1','col2']]`
- Select rows: `df.loc[index]` or `df.iloc[position]`
- Conditional selection: `df[df['Age'] > 25]`

Basic operations

- `df['NewCol'] = df['Age'] + 5` # add new column
- `df.drop('NewCol', axis=1, inplace=True)` # drop column
- `df.rename(columns={'Name': 'FullName'}, inplace=True)` # rename
- `df.sort_values('Age', ascending=False)` # sort
- `df.reset_index(drop=True, inplace=True)` # reset index

Data Cleaning Steps--

1. Handle Missing Values

- `df.isnull().sum()` # count nulls
- `df.dropna(inplace=True)` # drop rows with any null
- `df.fillna(0, inplace=True)` # fill null with 0
- `df['Age'].fillna(df['Age'].mean(), inplace=True)`
fill with mean

Data Cleaning Steps--

2.Handle Duplicates

- `df.duplicated().sum()` # how many duplicates
- `df.drop_duplicates(inplace=True)` # remove duplicates

Data Cleaning Steps--

3.Handle Data Types

- `df.dtypes` # check types
- `df['Age'] = df['Age'].astype(int)` # convert type
- `df['Date'] = pd.to_datetime(df['Date'])` # convert to datetime

Data Cleaning Steps--

4. Handle Outliers

- Detect:
- `df['Age'].describe()` # check min, max, quartiles
- Filter out:
- `df = df[df['Age'] < 100]` # remove unreasonable ages

Data Cleaning Steps—

5. String Operations

- `df['City'] = df['City'].str.strip()` # remove whitespace
- `df['City'] = df['City'].str.lower()` # to lowercase
- `df['City'] = df['City'].str.replace('mumbai','Mumbai')`

Data Cleaning Steps—

6. Encoding Categorical Variables

One-hot encoding

```
pd.get_dummies(df['City'])
```

Label encoding

```
from sklearn.preprocessing import LabelEncoder
```

```
le = LabelEncoder()
```

```
df['City_encoded'] = le.fit_transform(df['City'])
```

Data Cleaning Steps—

7. Handling Large Datasets

- Use `df.info(memory_usage='deep')` to check memory usage.

- Load partially:

```
pd.read_csv('data.csv', nrows=1000)
```

- Chunk loading:

```
for chunk in pd.read_csv('data.csv',  
    chunksize=5000):
```

```
    process(chunk)
```


Data Analysis with Pandas

- Summary statistics: `describe()`, `mean()`, `sum()`
- Grouping: `groupby()`
- Sorting: `sort_values()`
- Merging/Joining: `merge()`, `concat()`

Visualization

- Basic plotting: `df.plot()`
- Histogram: `df['col'].hist()`
- Integration with Matplotlib and Seaborn for advanced plots

Summary

- Pandas simplifies data handling in Python
- Provides powerful data structures and functions
- Essential for data science and machine learning projects